PLACEWISE — Exact Product Workflow and Technically Feasible MVP

Bro, now PLACEWISE has a clear product definition. The key is that **the user should not have to learn interior design, select every object manually, or describe a floor plan.** They should be able to upload one photo, say what they want to keep or remove in Tanglish, and receive a visual rearrangement of that same space.

PLACEWISE — exact product definition

An image-first, voice-controlled space rearrangement assistant for everyday people.

The product has two simultaneous outputs:

- **Rearranged image:** an edited version of the original photograph, with unwanted objects removed and retained objects visually repositioned.
- **Arrangement plan:** an annotated version of the photograph showing detected objects, their proposed positions, movement arrows and labels.

Both outputs should come from the same arrangement decision—not two separate AI guesses.

1. Exact user workflow

1. Upload or capture a photograph

The user uploads a room, workspace, kitchen, bedroom, shop, classroom or other physical-space image. No measurements or floor-plan drawing required.

2. Give a natural voice instruction

The user taps the microphone and speaks in Tanglish, Tamil, English or a supported language.

- **Removal-first:** “Indha rendu chairs-um table-um remove pannunga.”
- **Keep-first:** “Sofa, TV, fan, bookshelf ellame venum.”

The system must support both. If the user lists only unwanted objects, everything else is considered a keep candidate.

3. Detect and identify objects

The system identifies objects in the photograph and assigns each one a visible label and unique ID. Example: `chair_1`, `chair_2`, `sofa_1`, `table_1`, `tv_1`.

The user can correct a mistaken detection before proceeding. This is important because two similar chairs must not be confused with one another.

4. Resolve the instruction into object actions

The system maps the user's words to detected objects and creates a structured plan.

Object	Action
Sofa	KEEP
TV	KEEP
Bookshelf	KEEP
Chair 1	REMOVE
Chair 2	REMOVE
Coffee table	REMOVE
Other detected items	KEEP, unless clarified

If the user says something ambiguous—such as “that one”—the app asks them to tap the relevant object instead of guessing.

5. Generate a proposed arrangement

A spatial planner determines which retained objects can plausibly move, which should remain fixed, and where they could go.

The user sees the proposed arrangement before accepting it. They can optionally mark an object as fixed or request another arrangement.

6. Show both visual outputs

- **Edited photograph:** visual preview of the proposed result.
- **Placement plan:** object labels, proposed positions and movement arrows.

One important behavior: the user can choose *only things to remove* or *only things to keep*. PLACEWISE must not force them to enumerate everything.

2. Technical architecture

For a two-person team and a short challenge deadline, I recommend a modular pipeline rather than trying to build one giant multimodal AI system.

PLACEWISE processing pipeline

- **React + TypeScript frontend** — photo upload, microphone, object confirmation and result viewer.
- **FastAPI backend** — creates and manages each arrangement job.
- **Voice pipeline — Gnani Prisma v2.5** — audio to Tanglish/Tamil/English transcript.
- **Vision pipeline — YOLO + segmentation** — photo to detected objects and masks.
- **Instruction interpreter** — maps words to object IDs and creates KEEP / REMOVE / MOVE / FIXED actions.
- **Spatial arrangement engine** — object positions, collision checks, movement constraints and layout scoring.
- **Image editor** — inpainting, compositing and visual reconstruction.
- **Plan renderer** — masks, destination outlines, labels and arrows.
- **Two coordinated results** — edited photograph and annotated arrangement plan.

Recommended technologies

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS
Backend	Python FastAPI
Speech recognition	Gnani Prisma v2.5
Spoken responses	Gnani Timbre v2.5
Object detection	YOLO
Object segmentation	SAM 2 or compatible segmentation model
Image operations	OpenCV, Pillow
Inpainting	Image-editing/inpainting model
Spatial geometry	Shapely
Optional optimization	OR-Tools
Storage	Local storage initially; PostgreSQL/object storage later

3. The most important engineering decision: separate the plan from the image edit

This is where PLACEWISE can either become a convincing product or a very unreliable AI demo.

Do not ask an image-generation model to independently decide where everything goes. It might remove the wrong chair, change the sofa, invent a window, or move furniture in a physically impossible way.

Instead, create one canonical arrangement plan and use it to generate both outputs.

A. Object detection and segmentation

For the MVP, detect common object categories:

- **Furniture:** sofa, chair, table, bed, cupboard, shelf, desk.
- **Appliances:** TV, refrigerator, washing machine, fan.
- **Other household objects:** lamp, plant, basket, boxes and selected clutter.

Use YOLO for initial object detection and SAM 2 or another segmentation model to produce object masks.

Each detected object gets a record:

```
{
  "object_id": "chair_02",
  "class": "chair",
  "confidence": 0.94,
  "bbox": [420, 280, 610, 590],
  "mask": "chair_02_mask.png",
  "action": "REMOVE"
}
```

The confidence and bounding box are illustrative. Your actual system must obtain these from the vision pipeline.

Important: A bounding box is not enough for clean removal. You need a segmentation mask so the editor can distinguish the chair from the floor and nearby furniture.

B. Tanglish instruction interpretation

Use Prisma to transcribe the audio, then a separate instruction parser to produce structured actions.

Example input:

“Sofa venum, rendu chairs vendaam. TV and bookshelf irukkattum. Matha things neat ah arrange pannunga.”

Expected interpretation:

```
{
  "mode": "KEEP_AND_REMOVE",
  "keep": ["sofa", "tv", "bookshelf"],
  "remove": ["chair"],
  "rearrange_remaining": true,
  "unmentioned_objects": "KEEP",
  "clarification_required": false
}
```

The parser must resolve *which* chairs the user means. If two chairs are detected, both can be removed when the user says “rendu chairs.” If the user says “andha chair,” ask them to select it on the image.

User instruction	System behavior
Only says what to remove	Keep all other detected objects
Only says what to keep	Remove nothing unless explicitly requested; arrange the kept objects
Says keep and remove	Apply both lists
Says arrange everything	Keep all objects and propose new positions
Mentions an ambiguous object	Ask the user to tap it
Contradictory instructions	Ask which instruction takes priority

Do not silently interpret an unmentioned object as unwanted.

C. The spatial arrangement engine

This component is responsible for the *actual arrangement decision*.

For every retained object, maintain:

- Current position and estimated footprint.
- Proposed destination.
- Whether the object can move.
- Whether it must remain fixed.
- Whether the new position conflicts with another object.
- Confidence in the proposed position.

For the first MVP, use a 2D image-coordinate representation rather than attempting complete 3D reconstruction.

A proposed layout can look like:

```
{
  "objects": [
    {
      "id": "sofa_01",
      "action": "KEEP",
      "from": [120, 250],
      "to": [100, 220],
      "fixed": false
    },
    {
      "id": "tv_01",
      "action": "KEEP",
      "from": [650, 200],
      "to": [620, 200],
      "fixed": true
    },
    {
      "id": "chair_02",
      "action": "REMOVE"
    }
  ],
  "constraints": {
    "no_object_overlap": true,
    "stay_inside_visible_room": true,
    "preserve_fixed_objects": true
  }
}
```

Shapely can perform geometric overlap checks. OR-Tools can be added later for optimization. Initially, a deterministic rule-based placement algorithm is sufficient.

A necessary limitation: one ordinary photograph does not reveal exact room dimensions, object depth or all occluded surfaces. Therefore, a single-photo layout is a visual suggestion, not a physically measured floor plan. Avoid claiming that it guarantees real-world clearance.

D. Image editing and inpainting

This is the technically hardest part.

You need two separate operations:

- **Removal:** remove unwanted objects using their segmentation masks and reconstruct the background.
- **Rearrangement:** visually reposition retained objects while attempting to preserve their original appearance.

A feasible MVP strategy:

- Use inpainting to remove unwanted objects.
- Use segmentation masks to extract retained objects.

- Use compositing to reposition those objects on the cleaned image.
- Use image editing/inpainting to blend edges and reconstruct occluded regions.
- Keep the original image available so the user can compare the result.

For example, moving a sofa exposes floor and wall regions that were hidden behind it. Simple cut-and-paste cannot reconstruct these correctly. An image-editing model can help, but may alter furniture details or room features.

Level 1 — Reliable MVP (build this first)

- Remove unwanted objects with masks and inpainting.
- Keep retained objects in their original positions or make only simple, safe adjustments.
- Show destination outlines and arrows for proposed moves.
- Clearly label the result as a proposed arrangement.

Level 2 — Advanced visual rearrangement

- Extract and reposition retained objects.
- Reconstruct newly exposed backgrounds.
- Blend the edited objects into the photograph.
- Preserve visual identity as much as possible.

This distinction matters: an arrangement overlay can be geometrically controlled; a photorealistic rearranged image is probabilistic. Do not let an attractive edited image imply that a physically validated plan has been produced.

4. How the two outputs stay consistent

The spatial planner should create one saved `arrangement_plan.json`.

The image editor and plan renderer both consume that same file.

- **Canonical arrangement plan:** object IDs, actions, source masks, destination coordinates and constraints.
- **Image editing:** remove objects, reposition retained objects and blend the result.
- **Plan rendering:** draw destination outlines, labels and arrows.

The plan renderer should show:

- Green outline: retained object.
- Red or grey marker: removed object.
- Dashed outline: proposed destination.
- Arrow: movement direction.
- Object label: sofa, chair, table, etc.

These are visual conventions you can define in the app.

5. MVP features: what to build and what to postpone

With roughly ten days until the challenge deadline, you need a convincing end-to-end experience, not a complete interior-design platform.

Feature	MVP priority
Upload room photo	Must have
Tanglish voice input	Must have
Prisma transcription	Must have
Keep/remove instruction parsing	Must have
Detect and label objects	Must have

Feature	MVP priority
User correction of detected objects	Must have
Remove unwanted objects	Must have
Annotated arrangement plan	Must have
Rearranged image preview	Must have, with honest quality limitations
Download both outputs	Must have
Timbre spoken response	Nice to have
Multiple alternative layouts	Later
3D room reconstruction	Later
AR furniture placement	Later
Exact measurements	Later
Shopping/catalogue integration	Later

My recommendation: support furniture and general household objects, but make furniture the primary tested category. You can demonstrate both object types without promising reliable detection of every object in existence.

6. Backend API design

Keep the API small and predictable.

Endpoint	Purpose
`POST /api/rooms`	Upload photograph and create a job
`POST /api/rooms/{id}/transcribe`	Transcribe voice instruction
`POST /api/rooms/{id}/detect`	Detect and segment objects
`POST /api/rooms/{id}/instructions`	Parse keep/remove instructions
`PATCH /api/rooms/{id}/objects`	Correct object labels or actions
`POST /api/rooms/{id}/arrange`	Generate proposed layout
`POST /api/rooms/{id}/render`	Generate both visual outputs
`GET /api/rooms/{id}/results`	Fetch images and arrangement plan

Suggested project structure

```
placewise/
■■■ frontend/
■   ■■■ src/
■       ■■■ components/
■       ■   ■■■ ImageUploader.tsx
■       ■   ■■■ VoiceRecorder.tsx
■       ■   ■■■ ObjectCanvas.tsx
■       ■   ■■■ ObjectEditor.tsx
■       ■   ■■■ ResultViewer.tsx
■       ■■■ pages/
■
■■■ backend/
■   ■■■ app/
■   ■   ■■■ main.py
■   ■   ■■■ routes/
■   ■   ■   ■■■ rooms.py
■   ■   ■   ■■■ voice.py
■   ■   ■   ■■■ vision.py
■   ■   ■   ■■■ arrangement.py
■   ■   ■■■ services/
■   ■   ■   ■■■ gnani_stt.py
■   ■   ■   ■■■ instruction_parser.py
■   ■   ■   ■■■ object_detector.py
■   ■   ■   ■■■ segmentation.py
■   ■   ■   ■■■ layout_engine.py
■   ■   ■   ■■■ inpainting.py
■   ■   ■   ■■■ plan_renderer.py
■   ■   ■■■ schemas/
■   ■■■ requirements.txt
■
■■■ README.md
```

For the first version, you can store images and JSON files locally. Don't spend your limited time building a complex database. Add PostgreSQL when you actually need persistent user sessions or multiple saved rooms.

7. Gnani integration: make it meaningful

Gnani should be central to the user experience, not just a model name added to your submission.

Prisma v2.5 — the core integration

- Converts natural Tanglish speech into text.
- Handles regional-language and code-mixed instructions.
- Allows the user to speak naturally instead of navigating complex menus.

Gnani describes Prisma v2.5 as supporting Indian-language speech recognition and code-mixed input, including Tanglish. Its hosted API documentation should be your integration reference.

Timbre v2.5 — optional voice feedback

Use it to provide short responses such as:

- “Rendu chairs remove panna poren.”
- “Sofa and TV same place-la irukkum.”
- “Indha object identify panna mudiyala. Please tap pannunga.”

The voice feedback should be optional. Don't delay your main product while integrating TTS.

One practical note: verify the exact model version, endpoint, authentication and available credits in your challenge account before coding against an assumed API interface. Public documentation can describe a different version from the one provisioned for the challenge.

8. Ten-day execution plan

Suggested division of work for a two-person team.

Days 1–2: Build the foundation

Developer A: FastAPI upload endpoint, YOLO detection, segmentation and masks.

Developer B: React interface, image upload, microphone capture and results layout.

Deliverable: upload a photo and see detected objects.

Days 3–4: Voice and intent

Developer A: Object-to-action mapping and structured instruction parser.

Developer B: Prisma integration, transcript display and Tanglish test instructions.

Deliverable: voice instructions correctly identify which objects to keep or remove.

Days 5–6: Arrangement and removal

Developer A: Rule-based layout engine, geometric checks and inpainting pipeline.

Developer B: Object correction interface, labels, destination outlines and arrows.

Deliverable: a valid arrangement plan and visible object removal.

Days 7–8: Integrate both outputs

Developer A: Image editing, object compositing and result consistency.

Developer B: Before/after viewer, plan viewer, error handling and optional Timbre.

Deliverable: one complete user journey from photograph to two results.

Days 9–10: Test and submit

- Test multiple rooms and different object types.
- Test removal-first and keep-first Tanglish instructions.
- Fix object-selection and image-editing failures.
- Record a clean 60-second demonstration.
- Prepare the public repository, README and submission explanation.

9. How we prove PLACEWISE actually works

Don't evaluate it only by whether the final image looks attractive.

Build a test set of around 20–30 photographs and 50–100 voice instructions, including variations in Tanglish.

Evaluation	What to measure
Speech recognition	Correctly transcribed object names
Intent extraction	Correct KEEP/REMOVE classification
Object identification	Correct matching to the visible object
Segmentation	Whether masks cover the intended objects
Removal	Whether the unwanted object disappears cleanly
Arrangement	Overlap and room-boundary violations
Visual preservation	Whether retained objects remain recognizably the same
User experience	Whether users understand and can act on the proposed plan

Set targets before testing, then report actual results. Don't claim a 95% accuracy figure unless your test data demonstrates it.

10. Biggest risks and how to control them

Risk	Practical solution
Tanglish words transcribed incorrectly	Show transcript and allow editing
Multiple identical objects	Give each object an ID and let users tap it
Incorrect object detection	Allow manual add, remove and correction
Inpainting changes the room	Keep the original and provide an undo/compare option
Furniture appears to float	Use conservative movement and floor-contact checks
Impossible arrangement	Reject layouts with geometric conflicts
Single image lacks depth information	Clearly label the output as an approximate visual plan
Image editor changes furniture identity	Preserve source masks and compare the output with the original

11. What I would build first

I would make one scenario exceptionally good before expanding the scope.

First demonstration scenario: a cluttered living room

User says:

“Indha two chairs and coffee table vendaam. Sofa, TV, bookshelf venum. Remaining things neat ah arrange pannunga.”

PLACEWISE must:

- Identify the two chairs and coffee table.
- Understand that they should be removed.
- Preserve the sofa, TV and bookshelf.
- Propose a sensible arrangement for retained objects.
- Produce the edited photograph and annotated plan.

This scenario gives you a focused way to test speech, object detection, object references, removal, arrangement and image editing in one demo.

My final technical recommendation

Build PLACEWISE around three genuinely distinct capabilities:

- **Understand:** Gnani speech recognition plus structured Tanglish instruction parsing.
- **Decide:** Computer vision plus a deterministic spatial planner.
- **Visualize:** Image removal/reconstruction plus an annotated placement plan.

The first two should be dependable. The third should be visually impressive without pretending to be more physically accurate than the photograph allows.

The important product differentiator is not simply “AI interior design.” It is the ability to tell a system, in ordinary regional-language speech, what you don't want in your space—and have it understand the objects in your actual photograph and show you what to do next.

One decision I'd make now: don't build a chatbot interface. Make the photograph the main interface, the voice instruction the main input, and the two visual results the main experience.

Useful references: [Gnani Prisma API](https://www.gnani.ai/speech-to-text-api) · [Gnani API documentation](https://docs.gnani.ai/api/introduction/introduction) · [SAM 2](https://github.com/facebookresearch/sam2) · [Ultralytics YOLO](https://docs.ultralytics.com/).